

Natural Language Processing

(Spring 2026)

Assignment 1 Report

Student: *Zaeem Nasir 25L-8021*

Objective:

The main purpose of this Assignment 1 of the Natural Language Processing course is to understand and implement a complete pipeline of NLP, such as collecting text, cleaning it, extracting patterns using regex, tokenization, stopwords, stemming or lemmatization, POS, NER, building N-gram language models, unigram/bigram with and without smoothing and then finding perplexity. Implementing BPE for Urdu text and then building embedding models.

The assignment is divided into 4 equal parts with each has tasks in it.

PART 1

This part focuses on basic NLP preprocessing: HTML→text, regex extraction, tokenization, normalization, stopwords, corpus statistics, POS tagging, stemming vs lemmatization, and NER.

Dataset (Webpage Used)

URL Used: <https://www.fccollege.edu.pk/>

This page is my university, where I did my bachelors it contains information about departments, academics, and university related different sections, so it is fine for my token statistics and pattern extraction tasks

P1T1: HTML to Plain Text Utility

Goal

In this task, I created a function `fetch_clean_text(url)` to convert a webpage from HTML format into simple readable text. First, I downloaded the webpage

using `urllib.request` with a User-Agent header. Then, I used BeautifulSoup to remove HTML tags and extract only the visible text. I also removed extra spaces to make the text clean.

Output

- RAW HTML (first 500 chars):
- CLEAN TEXT (first 500 chars)

```
... RAW HTML (first 500 chars):
<!DOCTYPE html>
<html class="avada-html-layout-wide avada-html-header-position-top avada-is-100-percent-template" lang="en-US" prefix="og: http://ogp.me/ns# fb: h
<head>
  <meta http-equiv="X-UA-Compatible" content="IE=edge" />
  <meta http-equiv="Content-Type" content="text/html; charset=utf-8"/>
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <meta name="robots" content="index, follow, max-image-preview:large, max-snippet:-1, max-video-preview:-1" />

CLEAN TEXT (first 500 chars):
Welcome to Forman Christian College | FCCU Skip to content Departments/Offices Academics School of Management Faculty of Computer and Mathematica
```

P1T2: Regex Extraction + Tokenization

Goal

The goal of this task was to extract email addresses and phone numbers from the cleaned text using regular expressions (regex) before tokenization, because tokenization can break email patterns. After extracting contact information, the text was tokenized into sentences and words.

Output

- **Unique emails found:** []
- **Unique phone numbers found:** ['92 42) 99231581']
- **Total sentences:** 19
- **Total tokens:** 1051

```
Email regex: [a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}
Phone regex: (?:\+?\d{1,3}[\s-])?(?:\((\d{2,4})\)?[\s-])?\d{3,4}[\s-]?\d{3,4}

Unique emails: []
Unique phone numbers: ['92 42) 99231581']

Total sentences: 19
Total tokens: 1051
```

Explanation

Regex helps find patterns like emails/phones directly from raw text. After that, sentence tokenization splits text into sentences, and word tokenization splits it into tokens.

P1T3: Text Normalization

Goal

The goal of this task was to make the text clean, consistent, and easier to analyze. I converted all words to lowercase so that the same words are not treated differently due to capital letters. I removed extra spaces and line breaks to clean the text. I also replaced digits with the special token <NUM> so that different numbers are treated in the same way and do not increase the vocabulary size. Punctuation was handled carefully, keeping it only where necessary while focusing mainly on word tokens for analysis.

Output / Results

- Before normalization (first 300 chars)
- After normalization (first 300 chars)

```
BEFORE NORMALIZATION (first 300 chars):
```

```
Welcome to Forman Christian College | FCCU Skip to content Departments/Offices Academics School of  
Management Faculty of Computer and Mathematical Sciences Computer Science Mathematics Statistics  
Faculty of Education Education Health and Physical Education Faculty of Humanities English Mass  
Communic
```

```
-----  
AFTER NORMALIZATION (first 300 chars):
```

```
welcome to forman christian college | fccu skip to content departments/offices academics school of  
management faculty of computer and mathematical sciences computer science mathematics statistics  
faculty of education education health and physical edu
```

P1T4: Stopwords + Corpus Statistics

Goal

The goal of this assignment is to analyze the most frequent tokens in a text before and after removing stopwords, and to compare the results by reporting total token count, vocabulary size, and Type-Token Ratio (TTR) to observe changes in lexical diversity.

Results

Top-20 BEFORE stopwords (sample):

> (51), < (47), NUM (47), of (29), education (23), and (20), faculty (19), ...

Top-20 AFTER stopwords (sample):

NUM (47), education (23), faculty (19), student (18), center (17), office (15), ...

Statistics Summary

Metric	Before	After
Total Tokens	1156	774
Vocabulary Size	355	299
Type-Token Ratio (TTR)	0.3071	0.3863

P1T4 - STOPWORDS REMOVAL & CORPUS STATISTICS

Top-20 BEFORE Stopword Removal:

>	51
<	47
NUM	47
of	29
education	23
and	20
faculty	19
student	18
.	18
to	17
center	17
,	16
office	15
the	15
financial	14
fccu	12
university	11
forman	10
sciences	10
academic	10

Top-20 AFTER Stopword Removal:

NUM	47
education	23
faculty	19
student	18
center	17
office	15
financial	14
fccu	12
university	11
forman	10
sciences	10
academic	10
aid	10
rector	9
students	9
services	8
college	7
health	7
alumni	7
programs	7

CORPUS STATISTICS:

Metric	Before	After
Total Tokens	1156	774
Vocabulary Size	355	299
Type-Token Ratio (TTR)	0.3071	0.3863

Explanation

Stopwords (like *the*, *is*, *and*, *of*) are common but usually not meaningful for topic/keyword analysis. After removing them, vocabulary becomes more content-focused and TTR increases because the remaining words are more informative.

P1T5: POS Tagging & POS Distribution

Goal

The goal of this task is to perform Part-of-Speech (POS) tagging and identify and display the most frequent POS tags in the text, and extract example nouns and verbs to analyze the grammatical distribution of words.

Output / Results

Top 10 POS tags (by frequency):

- NN: 334
- JJ: 137
- NNS: 123
- VBP: 51
- NNP: 47
- VBG: 24
- VBD: 19
- VBZ: 10
- RB: 9
- VBN: 6

Example nouns (15):

['forman', 'college', 'fccu', 'skip', 'content', 'academics', 'school', 'management', 'faculty', 'computer', 'sciences', 'computer', 'science', 'mathematics', 'statistics']

Example verbs (15):

['urdu', 'pharmacy', 'faculty', 'fpssc', 'counseling', 'mercy', 'writing', 'student', 'fc', 'giving', 'alumni', 'chartered', 'dating', 'offers', 'university']

```

=====
P1T5 - POS TAGGING & POS DISTRIBUTION
=====

Top 10 POS Tags:

NN      334
JJ      137
NNS     123
VBP     51
NNP     47
VBG     24
VBD     19
VBZ     10
RB       9
VBN      6

-----

Example Nouns (15):

['forman', 'college', 'fccu', 'skip', 'content', 'academics', 'school', 'management', 'faculty', 'computer', 'sciences', 'computer', 'science', 'mathematics', 'statisti

Example Verbs (15):

['urdu', 'pharmacy', 'faculty', 'fpsc', 'counseling', 'mercy', 'writing', 'student', 'fc', 'giving', 'alumni', 'chartered', 'dating', 'offers', 'university']

-----

Full POS Distribution:

NN      334
JJ      137
NNS     123
VBP     51
NNP     47
VBG     24
VBD     19
VBZ     10
RB       9
VBN      6
RBR      3
JJ$      3
JJR      2
VB       2
CD       2
RP       1
PRP      1
=====

```

Explanation

POS tagging labels each word with its grammatical role (noun, verb, adjective, etc.). It helps in tasks like information extraction, parsing, and identifying important word types.

P1T6: Lemmatization vs Stemming

Goal

The goal of this task is to compare the effects of the Porter stemmer, Lancaster stemmer, and WordNet lemmatizer on a sample of 125 tokens to observe differences between stemming and lemmatization techniques

Output

Example rows from the produced table:

Token	Porter Stem	Lancaster Stem	Lemma
welcome	welcom	welcom	welcome
academics	academ	academ	academic
management	manag	man	management

P1T6 - STEMMING vs LEMMATIZATION COMPARISON			
Token	Porter Stem	Lancaster Stem	Lemma
welcome	welcom	welcom	welcome
forman	forman	form	forman
christian	christian	christian	christian
college	colleg	colleg	college
fccu	fccu	fccu	fccu
skip	skip	skip	skip
content	content	cont	content
academics	academ	academ	academic
school	school	school	school
management	manag	man	management
faculty	faculti	facul	faculty
computer	comput	comput	computer
mathematical	mathemat	mathem	mathematical
sciences	scienc	sci	science
computer	comput	comput	computer
science	scienc	sci	science
mathematics	mathemat	mathem	mathematics
statistics	statist	stat	statistic
faculty	faculti	facul	faculty
education	educ	educ	education
education	educ	educ	education
health	health	heal	health
physical	physic	phys	physical
education	educ	educ	education
faculty	faculti	facul	faculty
humanities	human	hum	humanity
english	english	engl	english
mass	mass	mass	mass
communication	commun	commun	communication
philosophy	philosophi	philosoph	philosophy
religious	religi	religy	religious
studies	studi	study	study
urdu	urdu	urdu	urdu
faculty	faculti	facul	faculty
natural	natur	nat	natural
sciences	scienc	sci	science
chemistry	chemistri	chem	chemistry
physics	physic	phys	physics
environmental	environment	environ	environmental
sciences	scienc	sci	science
pharmacy	pharmaci	pharm	pharmacy
kauser	kauser	kaus	kauser
abdulla	abdulla	abdull	abdulla
malik	malik	malik	malik
school	school	school	school
life	life	lif	life
sciences	scienc	sci	science
faculty	faculti	facul	faculty

Explanation

- Stemming cuts words roughly; it is fast but sometimes incorrect.
- Lemmatization returns dictionary form; it is more correct but needs linguistic rules.
- Lancaster is more aggressive than Porter, so it can shorten words too much.

P1T9: Named Entity Recognition (NER)

Goal

The goal of this task is to apply Named Entity Recognition to identify and extract named entities from the text under the categories of PERSON, ORGANIZATION, and GPE/LOCATION.

Output

- PERSON entities
- ORGANIZATION
- GPE/LOCATION

```
=====
P1T9 - NAMED ENTITY RECOGNITION (NER)
=====
```

```
PERSON Entities (Total: 0):
-----
```

```
ORGANIZATION Entities (Total: 1):
-----
```

```
NUM
-----
```

```
GPE / LOCATION Entities (Total: 0):
-----
=====
```

PART 2: Language Models and Smoothing

This part required training 4 models (unigram/bigram with smoothing) and evaluating perplexity on in-domain and out-of-domain corpora.

Dataset

- **Training:** Brown Corpus (NLTK)

- **Out-of-domain test:** Reuters Corpus (NLTK)

Preprocessing

- Added sentence markers: <s> at start and </s> at end (required).
- Lowercased tokens.
- Replaced rare words with <UNK> (threshold = 1) to reduce zero probability issues.

Training Summary

- Training sentences: **45,872**
- In-domain test sentences: **11,468**
- Out-of-domain test sentences: **54,716**
- Total training tokens: **1,020,759**
- Vocabulary size: **24,757**

Models Implemented

1. Unigram (unsmoothed)
2. Unigram (Laplace add-one smoothing)
3. Bigram (unsmoothed)
4. Bigram (Linear Interpolation smoothing)

“Unique / Good thing” I did here

- I computed perplexity in **log-space** (using log probabilities)

Sentence Generation Files

Generated 20 sentences each and saved:

- unigram_output.txt
- smoothunigram_output.txt
- bigram_output.txt
- smoothbigramLI_output.txt

Perplexity

Perplexity was computed for both test sets.

Model	In-domain PP	Out-domain PP
Unigram (unsmoothed)	725.90	945.00
Unigram (Laplace)	728.50	943.83

Bigram (unsmoothed)	inf	inf
Bigram (Linear Interp)	275.44	666.99

```

=====
PART 2 - DATA SUMMARY
=====
Training sentences: 45872
In-domain test sentences: 11468
Out-of-domain test sentences: 54716

Example training sentence:
['<s>', 'he', 'let', 'her', 'tell', 'him', 'all', 'about', 'the', 'church', '.', '</s>']

=====
PART 2 - TRAINING STATISTICS (after <UNK>)
=====
Total training tokens: 1020759
Vocabulary size: 24757

Top 10 most common words:
the          55897
,            46676
<s>          45872
</s>         45872
.            39479
of           29092
and          23140
to           20991
<UNK>        20399
a            18659

=====
PART 2 - MODELS TRAINED
=====
1) Unigram (unsmoothed)
2) Unigram (Laplace)
3) Bigram (unsmoothed)
4) Bigram (Linear Interp)

Saved 20 generated sentences to:
- unigram_output.txt
- smoothunigram_output.txt
- bigram_output.txt
- smoothbigramLI_output.txt

=====
PART 2 - PERPLEXITY RESULTS
=====
Unigram (unsmoothed) | In-domain PP: 725.90 | Out-domain PP: 945.00
Unigram (Laplace)    | In-domain PP: 728.50 | Out-domain PP: 943.83
Bigram (unsmoothed)  | In-domain PP: inf   | Out-domain PP: inf
Bigram (Linear Interp) | In-domain PP: 275.44 | Out-domain PP: 666.99

```

Answers to Required Questions (Part 2)

Q1) In unigram, what controls sentence length? How is it different from bigram?

- In unigram generation, each word is chosen independently. Sentence ends when `</s>` is sampled, so the probability of `</s>` controls average sentence length.
- In bigram generation, the next word depends on the previous word, so endings happen in a more context-based way and produce more structured sentences.

Q2) Do models assign drastically different probabilities? Why?

- Yes. Unsmoothed bigram often gives zero probability for unseen bigrams, making full sentence probability become zero (and perplexity becomes infinity).
- Smoothed models avoid this by giving small probabilities to unseen events.

Q3) Which model produces better / realistic sentences?

- The Smoothed Bigram (Linear Interpolation) produces more sentences than unigram because it uses context which is previous word and smoothing prevents dead ends.

Q4) Which test corpus has higher perplexity and why? (include values)

- Out-of-domain test has higher perplexity in working models because language and vocabulary style differs from training domain.
Example:
- Unigram unsmoothed: **945.00 (out) > 725.90 (in)**
- Bigram LI: **666.99 (out) > 275.44 (in)**

PART 3: Byte Pair Encoding for Urdu

This part does the BPE for Urdu segmentation and calculates the average characters per token for a self-collected Urdu corpus.

P3T1: BPE for Urdu Word Segmentation

Goal

Segment 50 Urdu sentences without spaces into meaningful subwords/words using BPE and a dictionary, and save the output.

Results

- Dictionary words: **5691**
- Sentences: **50**
- BPE merges learned: **500**
- Output saved: **50_segmented.txt**

```
.. Dictionary words: 5691
   Sentences: 50
   BPE merges learned: 500
   ✓ Saved output: 50_segmented.txt
```

P3T2: Average Characters per Word in Urdu Text

Goal: The goal of this task is to collect a large Urdu corpus at least 10,000 tokens, but I collected 12k+ tokens and then tokenized the text, calculated the total number of tokens and the average number of Unicode characters per token.

Urdu Corpus Sources

1. BBC Urdu live page: <https://www.bbc.com/urdu/live/ckg16vlk67nt?page=3>
2. UrduPoint kids story: <https://www.urdupoint.com/kids/detail/moral-stories/ajnabi-sardar-2432.html>

Results

- **Total tokens:** 12,654
- **Average Unicode characters per token:** 3.637

```
• Total tokens: 12654  
Average Unicode characters per token: 3.637
```

PART 4: Embedding Models

This part covers embedding models.

Datasets Used

- **Primary (unlabeled):** Reuters news-style dataset (4,000 docs used in notebook output)
- **Secondary (labeled):** AG News (train 120,000 / test 7,600)

P4T1: TF-IDF

Goal

The goal is to implement TF-IDF from scratch and report vocabulary size, sparsity ratio, OOV rate, and cosine similarity, while comparing results for word-level tokens vs BPE tokens.

Results

Word-level TF-IDF

- Vocab: **15000**
- Sparsity: **0.9817**
- OOV: **0.0343**
- Avg cosine similarity: **0.1651**

BPE-based TF-IDF

- Vocab: **186**
- Sparsity: **0.1067**
- OOV: **0.0**
- Avg cosine similarity: **0.8419**

```
..
---- P4T1 TF-IDF (FAST) ----
Vocab word: 15000 Sparsity: 0.9817 OOV: 0.0343
Vocab bpe : 186 Sparsity: 0.1067 OOV: 0.0
Avg cosine word: 0.1651
Avg cosine bpe : 0.8419
```

Analysis

BPE reduces vocabulary and almost removes OOV, so document vectors become less sparse.

P4T2: Count-Based Word Embeddings

Goal

Build a word-context co-occurrence matrix, apply PPMI, test with different windows, show nearest neighbors.

Results

- Window 2 co-occ pairs: **80000** (fast run)
- Example nearest neighbors shown for a word like **“president”** (neighbors printed in notebook)

```
---- P4T2 PPMI (FAST) ----
Window 2 | cooc pairs: 80000 | time: 0.56 sec
NN president : [('bush', 0.17227576634236536), ('female', 0.1683851897418059), ('official', 0.15902559510532285), ('emergency', 0.15425643820511475), ('as', 0.14906433811
NN company : [('eye', 0.17440198375606483), ('400', 0.1646040478355738), ('relief', 0.15986900277834667), ('patrol', 0.14125559249284547), ('visitors', 0.1274138195797612
NN war : [('nations', 0.16262062571188185), ('remain', 0.16061834795030427), ('performance', 0.14286398943995354), ('suggested', 0.1355161362801475), ('paid', 0.133951237
NN market : [('status', 0.26480727676412613), ('prices', 0.21594345185193548), ('policies', 0.19130780501662917), ('fly', 0.16862975604206532), ('stock', 0.16687140945189
Window 5 | cooc pairs: 80000 | time: 0.48 sec
NN president : [('bush', 0.4102062819032669), ('vice', 0.3175552484399176), ('snow', 0.28215844181911726), ('washington', 0.250614016757678), ('becoming', 0.2257028307598
NN company : [('th', 0.3433081946826451), ('employees', 0.31922347478164587), ('army', 0.29849175728264954), ('fort', 0.28190706870787924), ('unit', 0.2632235983324081)]
NN war : [('rose', 0.2697623648377676), ('broke', 0.2582442808969942), ('world', 0.2333283450419463), ('japan', 0.22594120338167423), ('against', 0.21867031246637128)]
NN market : [('given', 0.21678156686900438), ('mom', 0.21392615295629203), ('countries', 0.20247336089242002), ('aid', 0.18734193373396008), ('vehicles', 0.18489286175743
```

Explanation

PPMI highlights word-context relationships by down-weighting very common contexts and emphasizing informative co-occurrences.

P4T3: Neural Word Embeddings

Goal

The goal of this task is to implement SGNS (Word2Vec-style) from scratch without using embedding libraries, and report the training loss, training time, nearest neighbors, and a brief coverage/OOV discussion.

Results

- Device: CPU
- Vocab: **6000**
- Pairs: **40000**
- Epoch 1 loss: **4.1631**
- Train time: **0.61 sec**
- Example neighbors printed for “president”

```
---- P4T3 SGNS (FAST) ----
Vocab: 6000 Pairs: 40000
Epoch 1 loss=4.1631
Train time: 0.61 sec
president -> [('shocked', 0.47616270184516907), ('denver', 0.4604296386241913), ('aol', 0.43965235352516174), ('ballet', 0.4186064302921295), ('retu
company -> [('administration', 0.4478916525840759), ('voting', 0.44179561734199524), ('announcing', 0.4357191026210785), ('routes', 0.42770704627037
war -> [('wide', 0.5129271745681763), ('parliament', 0.4770171046257019), ('ships', 0.47398149967193604), ('mayor', 0.47231408953666687), ('rev', 0.
market -> [('equipment', 0.49685412645339966), ('essentially', 0.49400606751441956), ('chapter', 0.4758765697479248), ('operations', 0.4700500369071
Epoch 1 loss=4.1643
Train time: 0.63 sec
```

P4T4: Intrinsic Evaluation

Goal

The goal is to evaluate intrinsic embedding quality using custom word similarity pairs.

Results

- Usable pairs: **9 / 10**
- Spearman (SGNS-50): **-0.3737**

```
..
---- P4T4 Intrinsic ----
Usable pairs: 9 / 10
Spearman (SGNS-50): -0.3737
```

P4T5: Sentence Embeddings

Goal

The goal is to create sentence vectors using mean pooling and a TF-IDF weighted average, then evaluate their performance

Results

- Mean pooling accuracy: **0.51**
- TF-IDF weighted accuracy: **0.5183**

```
---- P4T5 Sentence similarity ----  
Mean pooling accuracy: 0.51  
TFIDF-weighted accuracy: 0.5183
```

P4T6: Extrinsic Evaluation

Goal

The goal is to use the learned embeddings or feature representations for a downstream task and evaluate accuracy and F1 score.

Results (AG News Classification)

- **TF-IDF Accuracy:** 0.8307
- **TF-IDF Macro-F1:** 0.8282
- **Sentence(mean) Accuracy:** 0.4473
- **Sentence(mean) Macro-F1:** 0.4407

```
---- P4T6 Extrinsic (FAST) ----  
TFIDF Accuracy: 0.8307 Macro-F1: 0.8282  
Sentence(mean) Accuracy: 0.4473 Macro-F1: 0.4407
```